

베이지스 튜터링 4주차

Jaewoo Shin

Oct 12 2024

Gibbs Sampler

Gibbs Sampler 의 경우에도 MCMC 의 알고리즘 중에서 한개 입니다. 다만, MH-Algorithm 와는 다르게 작동이 됩니다.

- Gibbs Sampler 의 경우 Multivariate 이면서 (모수가 여러개) Fully-Conditional Distribution 을 알 수가 있다면 사용이 가능합니다.
- 모수가 $\beta_1, \beta_2, \beta_3, \beta_4$ 으로 있다면, Posterior 분포는 $\pi(\beta_1, \beta_2, \beta_3, \beta_4 \mid X, Y)$ 과 같이 있습니다.
- 이것을 Fully-Conditional Distribution 으로 다음을 구할 수 있게 된다면 Gibbs Sampler 를 사용할 수 있게 됩니다.
 - $\pi(\beta_1 \mid \beta_2, \beta_3, \beta_4, X, Y)$
 - $\pi(\beta_2 \mid \beta_1, \beta_3, \beta_4, X, Y)$
 - $\pi(\beta_3 \mid \beta_1, \beta_2, \beta_4, X, Y)$
 - $\pi(\beta_4 \mid \beta_1, \beta_2, \beta_3, X, Y)$

Gibbs Sampler

Gibbs Sampler 의 경우 이전 슬라이드에서 구한 분포들을 세운 다음에, 해당 분포에서 뽑아서 값들을 업데이트 하는 방식으로 작동됩니다.

- Gibbs Sampler 의 경우 MH-Algorithm 와는 다르게 Acceptance Rate 이 1 입니다.
- 그렇기 때문에 $U \sim \text{Unif}(0, 1)$ 에서 뽑을 이유도 없게 됩니다.

$$\beta_1^{(t+1)} \sim \pi(\beta_1^{(t)} \mid \beta_2^{(t)}, \beta_3^{(t)}, \beta_4^{(t)}, X, Y)$$

$$\beta_2^{(t+1)} \sim \pi(\beta_2^{(t)} \mid \beta_1^{(t+1)}, \beta_3^{(t)}, \beta_4^{(t)}, X, Y)$$

$$\beta_3^{(t+1)} \sim \pi(\beta_3^{(t)} \mid \beta_1^{(t+1)}, \beta_2^{(t+1)}, \beta_4^{(t)}, X, Y)$$

$$\beta_4^{(t+1)} \sim \pi(\beta_4^{(t)} \mid \beta_1^{(t+1)}, \beta_2^{(t+1)}, \beta_3^{(t+1)}, X, Y)$$

Gibbs Sampler

알고리즘 :

- 1 Initialize $t = 0$, and $\mathbf{X}_0 = (X_1^{(0)}, \dots, X_d^{(0)})$
- 2 Sample by order

$$X_1^{(t+1)} \sim \pi \left(x_1 \mid X_2^{(t)}, X_3^{(t)}, \dots, X_d^{(t)} \right)$$

$$X_2^{(t+1)} \sim \pi \left(x_2 \mid X_1^{(t+1)}, X_3^{(t)}, \dots, X_d^{(t)} \right)$$

...

$$X_d^{(t+1)} \sim \pi \left(x_d \mid X_1^{(t+1)}, X_2^{(t+1)}, \dots, X_{d-1}^{(t+1)} \right)$$

- 3 Set $\mathbf{X}_{t+1} = (X_1^{(t+1)}, X_2^{(t+1)}, \dots, X_d^{(t+1)})$
- 4 Set $t = t + 1$ and go back to step 2 until $t = T$

Simple Linear Regression

단순한 선형회귀분석을 생각해보겠습니다. 그저 여러 개의 독립변수가 종속변수에 어떤 영향을 미치는지를 분석해주는 기법입니다.

$$y_i = \beta_0 + \beta_1 \cdot x_{i,1} + \beta_2 \cdot x_{i,2} + \cdots + \beta_p \cdot x_{i,p} + \epsilon_i, \quad \epsilon_i \stackrel{i.i.d}{\sim} N(0, \sigma^2)$$

$$y_i \stackrel{i.i.d}{\sim} N(\beta_0 + \beta_1 \cdot x_{i,1} + \beta_2 \cdot x_{i,2} + \cdots + \beta_p \cdot x_{i,p}, \sigma^2)$$

- X : 독립변수, 저희의 데이터라 생각해도 됩니다.
- Y : 종속변수, 저희가 궁금한 데이터 입니다.
- 회귀분석의 경우 그럼 회귀계수인 β_i 의 값을 찾는 것이 목표입니다.

Bayesian Regression

회귀분석에 베イズ 신념을 넣게 된다면 저희의 모수들을 확률변수로 바라봐야 합니다.

β_0, β_1 의 경우만 고려하면 다음과 같이 표현이 가능합니다.

$$y_i \mid \beta_0, \beta_1, \sigma^2 \stackrel{i.i.d}{\sim} \mathcal{N}(\beta_0 + \beta_1 \cdot x_{i,1}, \sigma^2)$$

$$\beta_0 \sim \mathcal{N}(\mu_0, \tau_0)$$

$$\beta_1 \sim \mathcal{N}(\mu_1, \tau_1)$$

$$\sigma^2 \sim \text{IG}(a, b)$$

Generalized Linear Model (GLM)

GLM 의 경우 이 Basic Linear Regression 의 Extended Version 이라고 생각하시면 됩니다.

선형회귀에서는 $\mathbf{Y} = \mathbf{X}\beta + \epsilon$, $\epsilon \stackrel{i.i.d}{\sim} N(0, \sigma^2)$ 으로 표현이 가능하며, 이때 종속변수의 평균은 다음과 같습니다.

$$E[\mathbf{Y} \mid \mathbf{X}] = \mu(\mathbf{X}) = \mathbf{X}\beta$$

이때 만약에 $\mathbf{Y} \mid \mathbf{X}$ 가 다른 분포를 따라가게 된다면 그것을 표현해주기 위해서 $\mathbf{X}\beta$ 의 형태를 바꿔줘야 합니다.

그것을 가능케하는 함수가 위에서는 $g(\cdot)$ 함수이며 이것을 **Link Function** 이라고 부릅니다.

Generalized Linear Model (GLM)

이것의 경우 Linear Regression 이 아닌 다른 Regression 에 도입이 가능하게끔 만드는 함수라고 생각하시면 됩니다.

$$g(E[\mathbf{Y} | \mathbf{X}]) = \mathbf{X}\beta$$

- Linear Regression : $g(\cdot)$ 함수는 Identity Function 이다.
- Binary Regression : $g(\cdot)$ 함수는 Logit Function 이다.
- Poisson Regression : $g(\cdot)$ 함수는 Log Function 이다.

GLM for Poisson Regression

$$\mathbf{Y} \mid \mathbf{X} \stackrel{i.i.d}{\sim} \text{Pois}(\mu(\mathbf{X}))$$

Poisson Regression 의 경우 Link Function 을 Log Function 으로 잡는다고 했습니다.

$$g(\mu(\mathbf{X})) = \log(\mu(\mathbf{X})) = \mathbf{X}\beta$$

$$\mu(\mathbf{X}) = e^{\mathbf{X}\beta}$$

그럼 여기에서 우도함수 (Likelihood) 를 다음과 같이 잡을 수 있습니다.

$$L(\beta) = \prod_{i=1}^n \frac{e^{-\mu(\mathbf{X}_i)} \cdot \mu(\mathbf{X}_i)^{Y_i}}{Y_i!}, \quad \mu(\mathbf{X}_i) = e^{\mathbf{X}_i'\beta}$$

이때 저희가 임의로 Prior 설정을 다음과 같이 하겠습니다.

$$\beta_j \sim N(0, 100), \quad \text{for } j = 1, \dots, p$$

Prior 와 Likelihood 를 둘다 구했기 때문에 이것을 통해서 MCMC 를 구축하면 됩니다.

GLM for Logistic Regression

$$\mathbf{Y} \mid \mathbf{X} \stackrel{i.i.d}{\sim} \text{Bernoulli}(\mu(\mathbf{X}))$$

Binary / Logistic Regression 의 경우 Link Function 을 Logit Function 으로 잡는다고 했습니다.

$$g(\mu(\mathbf{X})) = \log \left(\frac{\mu(\mathbf{X})}{1 - \mu(\mathbf{X})} \right) = \mathbf{X}\beta$$
$$\mu(\mathbf{X}) = \frac{e^{\mathbf{X}\beta}}{1 + e^{\mathbf{X}\beta}}$$

그럼 여기에서 우도함수 (Likelihood) 를 다음과 같이 잡을 수 있습니다.

$$L(\beta) = \prod_{i=1}^n \mu(\mathbf{X}_i)^{Y_i} \cdot (1 - \mu(\mathbf{X}_i))^{1-Y_i}, \quad \mu(\mathbf{X}_i) = \frac{e^{\mathbf{X}_i' \beta}}{1 + e^{\mathbf{X}_i' \beta}}$$

이때 저희가 임의로 Prior 설정을 다음과 같이 하겠습니다.

$$\beta_j \sim N(0, 100), \quad \text{for } j = 1, \dots, p$$

GLM Summary

결국에는 Regression 형태의 데이터는 Linear Regression 뿐만이 아니라 여러 형태가 존재합니다.

그 중에서 강의에서 사용될 것들 (그리고 자주 사용되는 것들) 은 다음과 같습니다.

- Linear Regression
- Logistic Regression
- Poisson Regression

이것을 GLM 을 활용시켜서 Regression Setting 으로 바꿔주는 것이 가능하며, Likelihood 역시 구축하는 것이 가능합니다.

이것에 걸맞는 Prior 를 세팅해주기만 한다면, Posterior 구축은 완성이며 이를 토대로 MCMC 를 진행해주시면 됩니다.

Other Libraries in R for MCMC

패키지 없이 진행하면 코드 짜는 것이 생각보다 어렵기 때문에 베이지에서는 다음과 같은 방법들을 활용합니다.

- nimble
- adaptMCMC
- RcppArmadillo (C++ 코드를 직접 짜야되는 방법)

Nimble

C++ 에서의 Compiler 를 가져와서 연산량과 효율성 측면에서 더욱 더 빠른 모습을 보여준다.

- 1 먼저 모델을 정의를 해준다.
- 2 사용할 초깃값들, 데이터들, 그리고 파라미터 값들을 가져와준다.
- 3 MCMC Sampling 을 진행해준다.

예시 문제

$$x_1, x_2, \dots, x_n \stackrel{i.i.d}{\sim} N(\mu, \sigma)$$

$$\mu \sim N(0, 10)$$

$$\sigma^2 \sim \text{Unif}(0, 10)$$

μ 와 σ 값들을 MCMC 를 통해서 구현해봐라.

데이터와 True 파라미터 값들은 다음과 같이 구현해서 진행해보라.

```
true_mu = runif(1, -10, 10)
true_sigma = rgamma(1, 3)
x = rnorm(1000, mean = true_mu, true_sigma)
```

Defining the Model

우선은 모델 자체를 먼저 정의해줘야 합니다.

```
model_glm <- nimbleCode(  
  # Data Model  
  for(i in 1:n)  
    X[i] ~dnorm(mean = mu, sd = sigma)  
  
  # Parameter Model  
  mu ~dnorm(mean = 0, sd = sqrt(10))  
  sigma ~dunif(0, 10)  
)
```

Setting Values

다음으로는 초깃값들, 데이터들 등을 설정해줘야 합니다.

```
niter <- 1e5  
consts <- list(n= length(x))  
data <- list(X = x)  
inits <- list(mu = 2, sigma = 2)
```


Running NimbleMCMC

마지막으로는 그저 nimbleMCMC 를 돌려주면 됩니다.

```
samples_glm <- nimbleMCMC(model_glm, data = data,  
  inits = inits, constants=consts,  
  monitors = c("mu", "sigma"), samplesAsCodaMCMC=TRUE,  
  WAIC=FALSE,summary=FALSE, niter = niter,  
  nburnin = 0, thin=1, nchains = 1)
```

Gibbs Sampler Problem

다음과 같은 Linear Regression 상황이라고 두겠습니다. 이때 저희의 데이터인 $\{x_i\}_{i=1}^n$ 의 경우 `Gibbs_Tutoring.RData` 에 있습니다.

$$y_i = \beta_0 + \beta_1 \cdot x_i + \epsilon_i, \quad \epsilon_i \stackrel{i.i.d}{\sim} N(0, \sigma^2)$$

이때 우리의 모수들은 $\beta_0, \beta_1, \sigma^2$ 이며 Prior 분포는 다음과 같이 두겠습니다.

$$\begin{aligned}\beta_0 &\sim N(0, 10) \\ \beta_1 &\sim N(0, 10) \\ \sigma^2 &\sim \text{IG}(0.01, 0.01)\end{aligned}$$

각 모수들의 Conditional Distribution 을 구한 다음에 Gibbs Sampler 를 이용해서 $(\beta_0, \beta_1, \sigma^2)$ 을 10,000 개 만큼 샘플해주세요.